Reversible Data Hiding in Encrypted Images Based on Multi-MSB Prediction and Huffman Coding

Zhaoxia Yin, *Member, IEEE*, Youzhi Xiang, and Xinpeng Zhang, *Member, IEEE*

***Abstract*—With the development of cloud storage and privacy protection, reversible data hiding in encrypted images (RDHEI) has attracted increasing attention as a technology that can: embed additional data in the image encryption domain, ensure that the embedded data can be extracted error-free, and the original image can be restored losslessly. In this paper, a high-capacity RDHEI algorithm based on multi-MSB (most significant bit) prediction and Huffman coding is proposed. At first, multi-MSB of each pixel was predicted adaptively and marked by Huffman coding in the original image. Then, the image was encrypted by a stream cipher method. At last, the vacated space can be used to embed additional data by multi-MSB substitution. Experimental results show that our method achieved higher embedding capacity while comparing with the state-of-the-art methods.**

***Index Terms*—Reversible data hiding, encrypted images, privacy protection, Huffman coding.**

I. Introduction

DATA hiding is a technique to embed data into multimedia without leaving any obvious distortion and the embedded data can be extracted error-free [1]. Moreover, for reversible data hiding (RDH), the original cover medium can be restored losslessly. Due to these features, RDH plays an important role in many multimedia applications, such as medical image processing [2], multimedia archive management [3], image trans-coding [4], and three-dimensional mesh models protection [5], etc.

Early RDH methods in digital images were realized by creating room for embedding additional data by using lossless compression [6]–[8]. In general, for given embedding payload, a good RDH method expects to minimize the distortion of the marked image leaving by data embedding. To achieve better rate-distortion performance, researchers have proposed many RDH methods in the past two decades [9]–[15]: mainly based on histogram shifting [9]–[12] and difference expansion [13]–[15]. Histogram shifting uses the peak points of image histogram to embed data, while difference expansion embeds data by expanding the difference between two pixels. These RDH methods obtain good rate-distortion performance but only can realize in the plaintext domain of images.

With the growing demand for privacy protection, multimedia signal processing in encrypted domain has greatly attracted researchers as the content owner's privacy remains protected [16]. Therefore, many researchers are devoted to developing reversible data hiding in encrypted images (RDHEI) [17]–[19], which can realize reversible data hiding and image content protection at the same time. RDHEI methods first encrypt the original image with an image encryption algorithm [20]–[22], then embed additional data in the image encryption domain while ensuring that the embedded data can be extracted error-free and the original image can be restored losslessly. As shown in Fig. 1, the existing RDHEI methods can mainly be divided into two categories: vacating room after encryption (VRAE) and reserving room before encryption (RRBE). In VRAE methods [23]–[27], a given cover image is encrypted at first and then additional data can be embedded by slightly modifying the encrypted pixels. Different from VRAE methods, RRBE methods [28]–[32] use the spatial correlation of the original image to reserve room for embedding data before image encryption.

Ma *et al.* [28] first preprocessed the original image to reserve room before encryption. Then the reserved room can be transferred to the encrypted image for embedding additional information. In [29], Zhang *et al.* reserved room before encryption based on an estimation technique. They estimated a small portion of the pixels through a large portion of pixels in the original image. Then additional data can be embedded by modifying the estimation errors. In another method of Zhang *et al.* [30], the original image is encrypted by a public key known to the receiver. Then additional information can be embedded into the encrypted image by multi-layer wet paper coding. In the method of Xu and Wang [31], a stream cipher is utilized to encrypt sample pixels and a specific encryption mode is designed to encrypt interpolation-error of non-sample pixels. Then additional data can be embedded into interpolation-error by traditional RDH techniques such as histogram shifting and difference expansion. Huang *et al.* [32] put forward a new simple yet effective framework for RDH in the encrypted domain. In this framework, the original image is encrypted by block permutation and pixel bit-level stream cipher to generate the encrypted image, while the embedding process is the same as the traditional RDH methods.

Manuscript received January 31, 2019; revised June 22, 2019; accepted August 3, 2019. Date of publication August 20, 2019; date of current version March 24, 2020. This work was supported by National Natural Science Foundation of China under Grants 61872003, U1636206, and 61860206004. The associate editor coordinating the review of this manuscript and approving it for publication was Prof. Marco Grangetto. *(Corresponding author: Xinpeng Zhang.)*

Z. Yin and Y. Xiang are with the Key Lab of Intelligent Computing and Signal Processing of Ministry of Education, School of Computer Science and Technology, Anhui University, Hefei 230601, China (e-mail: yinzhaoxia@ahu.edu.cn; 1875738190@qq.com).

X. Zhang is with the School of Computer Science, Fudan University, Shanghai 200433, China (e-mail: zhangxinpeng@fudan.edu.cn).

Color versions of one or more of the figures in this paper are available online at http://ieeexplore.ieee.org.

Digital Object Identifier 10.1109/TMM.2019.2936314

1520-9210 © 2019 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: National PingTung University -. Downloaded on May 26,2021 at 13:39:47 UTC from IEEE Xplore. Restrictions apply.

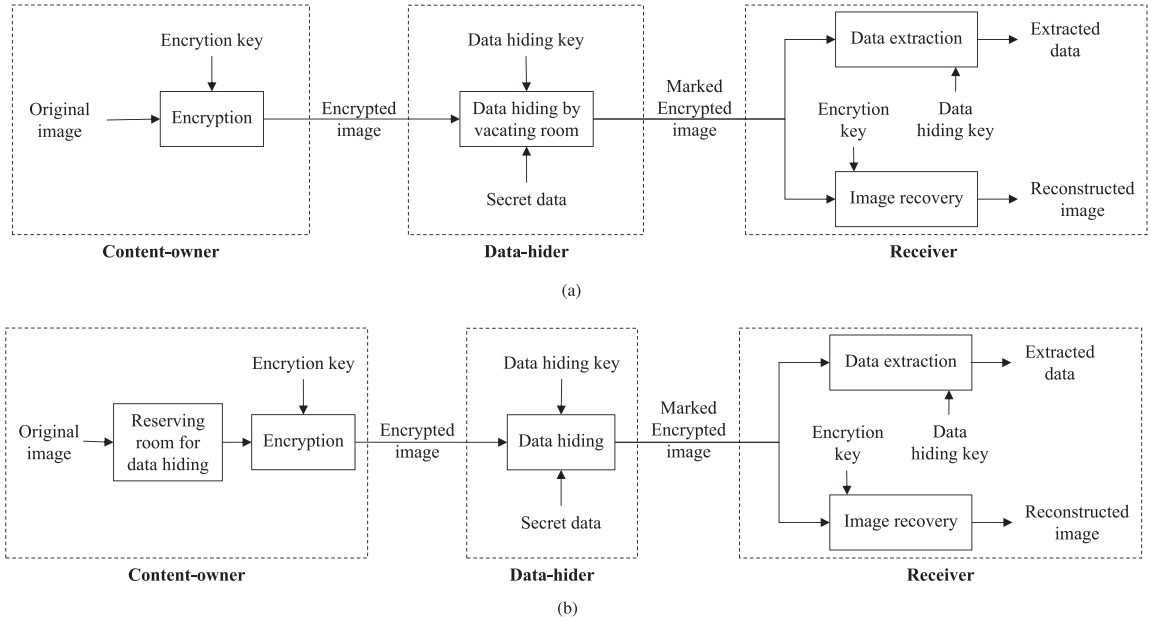


Fig. 1. Two different frameworks of RDHEI methods: (a) VRAE and (b) RRBE.

In order to improve the payload performance of RDHEI, i.e., embedding capacity (EC), several recent methods embed additional data by marking or compressing pixels of the original image [33]–[37]. Puteaux *et al.* [33] proposed an MSB prediction method to generate a label map of the original image and embed it into the encrypted image obtained by stream cipher encryption. Finally, additional data can be embedded in the encrypted image according to the label map. On this basis, Puteaux *et al.* [34] further proposed an improved RDHEI method, which marks multiple bit planes of the original image. With a similar idea, Puyang *et al.* [35] also proposed an improved RDHEI method. They used two MSBs to mark pixels of the original image so that the EC increases. In another method proposed by Yi *et al.* [36], a small number of pixels are used as reference values to calculate the prediction error of most pixels and a parametric binary tree labeling method is adopted to distinguish all prediction errors. In the data hiding stage, data can be embedded according to the label of pixels in the encrypted image. Different from the above methods, Chen *et al.* [37] proposed a bit plane compression algorithm. They compressed the MSB planes of images to generate room for high-capacity embedding. Although these five RDHEI methods have a great improvement in EC compared to that of the previous algorithms, [33]–[35] have a probability of generating errors in the decryption stage, while [36] and [37] does not make full use of the redundant space of the original image. Therefore, we propose a new high-capacity and error-free RDHEI method based on adaptive multi-MSB prediction and Huffman coding in this paper. Experimental results show that our method has better performance while comparing with the state-of-the-art methods [33]–[37].

The rest of the paper is organized as follows. Section II introduces the latest three related work. Section III describes the proposed method in detail. Section IV shows experimental results and analysis. Section V concludes this paper and explains the future work.

II. Related Work

In this chapter, we will detail the specific steps of three recent work, namely Puteaux *et al.*'s method [33], Yi *et al.*'s method [36], and Chen *et al.*'s method [37]. Puteaux *et al.* [33] only marked and embedded data on the MSB of the pixel. [34] and [35] are both improved on the basis of [33], and the main ideas are roughly the same. The difference is that they embedded additional information by multi-bit plane embedding and 2-MSB replacement. Yi *et al.* [36] proposed an RDHEI method using parametric binary tree labeling (PBTL), which keeps spatial correlations within small blocks and exploiting the spatial redundancy of the encrypted image to embed secret data. Chen *et al.* [37] took advantage of the spatial correlation of the original images to vacate room for embedding data by compressing the bit plane before image encryption.

A. Puteaux et al.'s Method [33]

1) MSB Prediction Error Detection: For a pixel $x(i,j)$ in the original image I with $m \times n$ pixels, where $1 \le i \le m$ and $1 \le j \le n$. Its inverse value is expressed as $inv(i,j) = (x(i,j) + 128) \bmod 256$. The predicted value $px(i,j)$ is computed through the left and top pixels of $x(i,j)$,

$$px(i,j) = \left\lfloor \frac{x(i-1,j) + x(i,j-1)}{2} \right\rfloor \tag{1}$$

where $\lfloor * \rfloor$ is the floor operations. Then the two absolute differences can be calculated and recorded as Δ and Δ^{inv}, that is:

$$\begin{cases} \Delta = |px(i,j) - x(i,j)| \\ \Delta^{inv} = |px(i,j) - inv(i,j)| \end{cases} \tag{2}$$

If $\Delta < \Delta^{inv}$, there is no prediction error because the original value of $x(i,j)$ is closer to its predictor than the inverse value, and the map of $x(i,j)$ is marked as '0', indicating it can embed

Authorized licensed use limited to: National PingTung University -. Downloaded on May 26,2021 at 13:39:47 UTC from IEEE Xplore. Restrictions apply.

data in the encrypted image. Otherwise, there is a prediction error and the map is marked as '1'.

2) Image Encryption and Pixel Marking: In this part, the original image I is encrypted through a stream cipher, then the map is embedded in the encrypted image by MSB substitution. Note that it should be done in blocks of 8 pixels when embedding the map, which aims to reduce the error rate during data extraction and image recovery. In addition, if there is at least one error pixel in a block, the MSB of pixels in the previous and the following blocks are replaced by '1' as two flags of the current erroneous block.

3) Data Hiding by MSB Substitution: Before embedding data, the to-be-inserted message is first encrypted by the data hiding key. Then the MSB of each pixel in the error-free block can be substituted by one-bit secret message.

4) Data Extraction and Image Recovery: On the receiving side, different results are obtained depending on the key that is owned. If the receiver only has data hiding key, the secret message can be obtained by extracting the MSB of each pixel in the correct block. If the receiver only has the encryption key, the marked encrypted image is directly decrypted to obtain the reconstructed image with the same 7 LSBs (least significant bits) as the original image, then the MSB can be recovered by prediction error detection method. So, only the receiver has two keys at the same time, data extraction and image recovery can be achieved at the same time.

B. Yi et al.'s Method [36]

1) PBTL: Suppose that all the pixels in an image are divided into two parts, namely G1 and G2. Then set two parameters α and β, where $1 \leqslant \alpha, \beta \leqslant 7$. For the pixels in G2, use β bits of all-zero binary code to mark. For G1, classified all pixels into n_α different sub-categories according to α and β, where n_α is calculated by:

$$n_\alpha = \begin{cases} 2^\alpha - 1 & , \quad \alpha \leqslant \beta \\ (2^\beta - 1) \times 2^{\alpha-\beta} & , \text{ otherwise} \end{cases} \tag{3}$$

For the pixel in different sub-categories, use different α bits of binary code to mark. For example, if $\alpha = 2$ and $\beta = 3$, one binary code '000' used to label pixels in G2, and three binary codes '01, 10, 11' used to label pixels in different sub-categories of G1.

2) Image Encryption and Pixel Grouping: Image encryption involves two processes: block permutation and pixel modulation. After obtaining the encrypted image, divide it into non-overlapped blocks and separate all pixels into four sets, namely reference pixels (P_r), one special pixel (P_s), embeddable pixel (P_e) and non-embeddable pixel (P_n). Here, the pixels in P_e and P_n are separated based on their prediction errors which are equal to the current pixel value minus the reference pixel value. If the prediction error e_i of current pixel satisfies following condition, the pixel is divided into P_e, conversely, the pixel is divided into P_n,

$$\left\lceil -\frac{n_\alpha}{2} \right\rceil \leqslant e_i \leqslant \left\lfloor \frac{n_\alpha - 1}{2} \right\rfloor \tag{4}$$

where $\lceil * \rceil$ and $\lfloor * \rfloor$ are the ceil and floor operations, respectively.

3) Pixel Labeling Using PBTL: After pixel grouping, the pixels in P_r and P_s remain unchanged, and only the pixels in P_e and P_n are marked using the PBTL method.

4) Data Hiding: In the final encrypted image, α and β are first stored in P_s, and the original 8 bits of P_s are stored as auxiliary information in P_e. In addition, for pixels in P_n, the original β-bits data before labeling needs to be recorded as auxiliary information. After pixel labeling, the original value of each pixel in P_e can be obtained through its α bits binary label and reference pixels, so that the remaining (8-α) bits of each pixel can embed the additional information by bit replacement. Of course, for data security, it is necessary to encrypt the secret data according to the data hiding key before embedding operation.

5) Data Extraction and Image Recovery: For the receiver, two parameters α and β can be extracted directly in P_s. Then the pixels can be grouped according to the labels of their front α or β bits. Next, the secret data can be obtained by extracting the remaining (8-α) bits of each pixel in P_e. In the image restoration process, one special pixel and the front β bits of pixels in P_n are restored based on the extracted auxiliary information, and the pixels in P_e are restored based on their labels and reference pixels. Finally, the original image is obtained by permuting all blocks and pixels inversely.

C. Chen et al.'s Method [37]

1) Rearrangement of the Bit Plane: The scheme first divides a bit plane of the image into non-overlapped blocks of the same size. Then all blocks are scanned through row by row or column by column, and all bits in each block are also scanned in this way. Therefore, there are four ways to generate a bitstream. The purpose of this is to get a bitstream of the bit plane that can be efficiently compressed.

2) Extended Run Length Coding: Given a bitstream, the adjacent repeating bits in the sequential scan is called a "run", and its length is denoted L. If $L < 4$, there is a fixed-length codeword to denote "run", which consists of a prefix and a fixed number of tail bits. The prefix is fixed to '0', and the tail bits are the L_{fix} bits symbol intercepted from the original bitstream, where L_{fix} is a predefined fixed number. If $L \geqslant 4$, there is a variable-length codeword to denote "run", which consists of a prefix, a length symbol and a tail bit. The prefix consists of $(L_{pre} - 1)$ sequential bits '1' and end with bit '0', where L_{pre} is calculated by $L_{pre} = \lfloor \log_2 L \rfloor$. The length symbol represents the value of l, where $l = L - 2^{L_{pre}}$, and its length is also L_{pre} bits. One tail bit indicates whether this "run" is all 0 or 1.

3) Vacating Room for Embedding: For an image, the rearrangement scheme scans the bit plane from MSB to LSB to generate bitstreams, and the coding scheme compresses the bitstream to vacate room. Of course, not all bit planes of an image can be compressed. Therefore, the method only compresses the front consecutive compressible bit planes. The vacated room of these compressible bit planes is used to embed LSBs, then additional information can be embedded in the LSBs.

4) Image Encryption and Data Embedding: After performing the above processing, a stream cipher is used to encrypt the image containing the embeddable room. Then the additional

Authorized licensed use limited to: National PingTung University -. Downloaded on May 26,2021 at 13:39:47 UTC from IEEE Xplore. Restrictions apply.

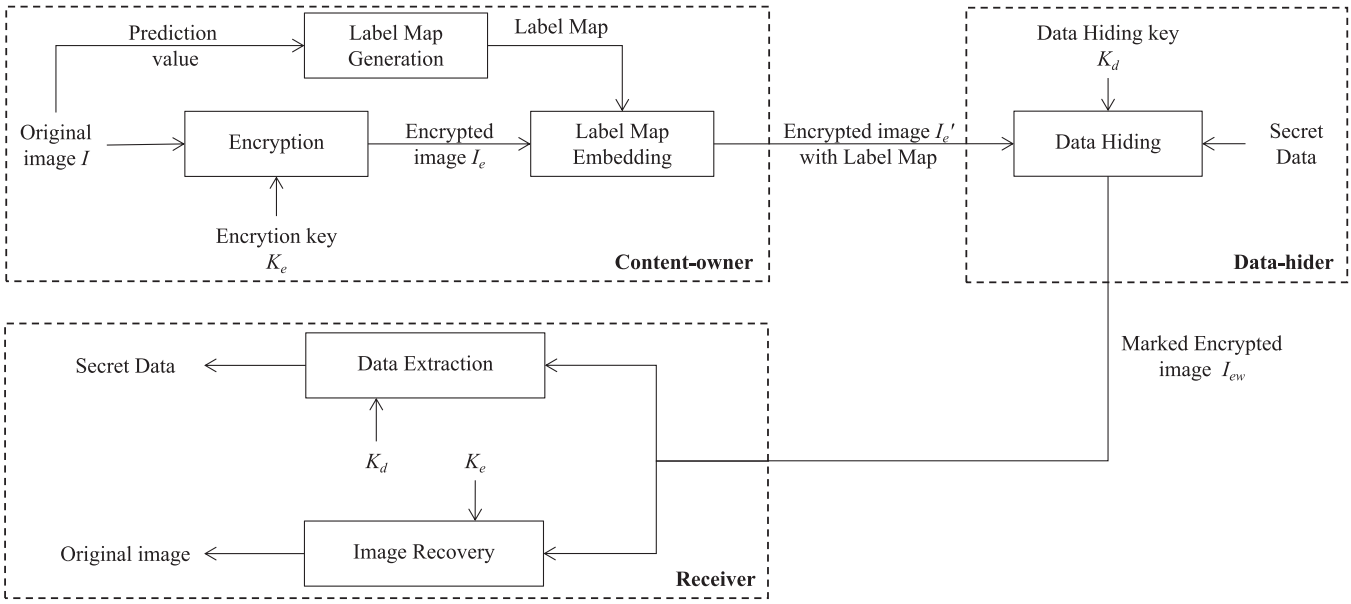


Fig. 2. The framework of the proposed method.

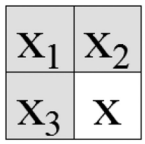


Fig. 3. The context of the current pixel by MED predictor.

information is encrypted and embedded in the vacated room by bit replacement.

5) Data Extraction and Image Recovery: When the receiver accepts the embedded encrypted image from the data hider, the receiver can directly extract the encrypted additional information and decrypt it by the key to obtain the original data. If the receiver has an encryption key at the same time, the image can be decrypted and the compressed bit planes can be decompressed to restore the original image.

III. Proposed Method

In this paper, we propose a new method with higher capacity and error-free data extraction and image decryption based on multi-MSB prediction and Huffman coding. As shown in Fig. 2, the proposed method contains three phases: 1) Image encryption is done by the content-owner; 2) Data embedding is done by the data-hider; 3) Data extraction and image decryption is done by the receiver. In the image encryption phase, we first calculate a label map of the original image before image encryption and embed the label map into the encrypted image. In the embedding phase, we can embed multi-bit information in each encrypted pixel by adaptive multi-MSB substitution according to the label map. Finally, for a receiver having data hiding key or image encryption key, data extraction or image recovery can be performed separately, and the extracted data and the restored image are both error-free.

A. Label Map Generation

For the original image I of size $m \times n$, the predicted value of each pixel is first calculated. As shown in Fig. 3, we calculate the predicted value $px(i,j)$ using the median edge detector (MED) predictor [6] based on three pixels around the current pixel $x(i,j)$, in which $1 \le i \le m$ and $1 \le j \le n$, the formula is as follows:

$$px = \begin{cases} \max(x_2, x_3) & , \ x_1 \le \min(x_2, x_3) \\ \min(x_2, x_3) & , \ x_1 \ge \max(x_2, x_3) \\ x_2 + x_3 - x_1 & , \ \text{otherwise} \end{cases} \tag{5}$$

Next, we convert the values of $x(i,j)$ and $px(i,j)$ into an 8-bit binary sequence by Eq. (6), denoted as $x^k(i,j)$ and $px^k(i,j)$, where $k = 1, 2, \ldots, 8$.

$$x^k(i,j) = \left\lfloor \frac{x(i,j) mod\, 2^{9-k}}{2^{8-k}} \right\rfloor, \ k = 1, 2, \ldots, 8 \tag{6}$$

Then, compare each bit of $x^k(i,j)$ and $px^k(i,j)$ sequentially from MSB to LSB until a certain bit is different, and the current pixel's label is equal to the length of their same bits. Since the converted binary sequence has 8 bits, the label of the pixel has 9 cases, namely from 0 to 8. Assuming that the tag value is represented by t, i.e. $t = 0, 1, 2, \ldots, 8$ and the maximum value of t obtained according to Eq. (7) is the label of the current pixel $x(i,j)$,

$$\begin{aligned} &arg \max_{t} \quad x(i,j)^{tMSB} = px(i,j)^{tMSB}, \ t = 0, 1, 2, \ldots, 8 \\ &\text{subject to} \begin{cases} x(i,j)^{tMSB} = x(i,j) - (x(i,j) mod\, 2^{8-t}) \\ px(i,j)^{tMSB} = px(i,j) - (px(i,j) mod\, 2^{8-t}) \end{cases} \end{aligned} \tag{7}$$

where $x(i,j)^{tMSB}$ and $px(i,j)^{tMSB}$ are the t-MSB (front t MSBs of the pixel) values of $x(i,j)$ and $px(i,j)$, respectively.

Authorized licensed use limited to: National PingTung University -. Downloaded on May 26,2021 at 13:39:47 UTC from IEEE Xplore. Restrictions apply.

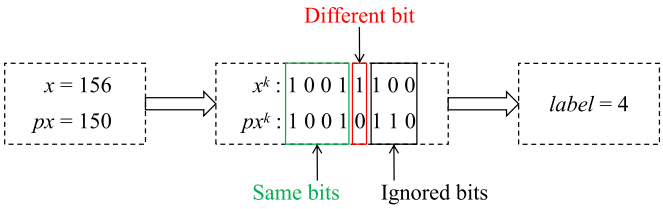


Fig. 4. An example of pixel labeling.

The tag value t of the current pixel is obtained by Eq. (7), and it means that the pixel can embed $(t+1)$ bits in the subsequent data hiding process. The reason is that the front $(t+1)$ MSBs of the original pixel can be obtained from its predicted value. In other words, the front t-MSB of the original pixel is the same as its predicted value, and the $(t+1)^{th}$ MSB can be obtained by negating the value of the corresponding position of its predicted value.

For example, as shown in Fig. 4, assume that the current pixel value x is equal to 156 and its predicted value px is equal to 150. Then x and px are converted into the 8-bit binary sequences, denoted x^k and px^k $(k = 1, 2, \ldots, 8)$, respectively, i.e. $x^k =$ '10011100', $px^k =$ '10010110'. By comparison, it can be obtained that the sequence of x^k and px^k are different in the fifth position, that is, the first four bits are the same, so the label of the pixel x is '$label = 4$', that is, 5 bits of information can be embedded in this pixel.

Finally, we can scan all the pixels in the image by the above method to obtain the label map of the original image. Note that the pixels in the first row and column are reference pixels and not marked.

B. Image Encryption

In this part, each pixel of the original image is encrypted by using an encryption key K_e. First, we generate a pseudo-random matrix r of size $m \times n$ through the key K_e. Next, convert the current pixel $x(i,j)$ and its corresponding $r(i,j)$ into the 8-bit binary sequence according to Eq. (6), denoted as $x^k(i,j)$ and $r^k(i,j)$. Then, the following encryption operation is performed,

$$x_e^k(i,j) = x^k(i,j) \bigoplus r^k(i,j),\ k = 1, 2, \ldots, 8 \tag{8}$$

where $x_e^k(i,j)$ is the encrypted 8-bit binary sequence, and $\bigoplus$ is the exclusive-or (XOR) operation. Finally, the encrypted pixel $x_e(i,j)$ can be calculated by Eq. (9). In this way, we get the encrypted image I_e.

$$x_e(i,j) = \sum_{k=1}^{8} x_e^k(i,j) \times 2^{8-k},\ k = 1, 2, \ldots, 8 \tag{9}$$

C. Huffman Coding

According to the label map of the original image obtained from Section III-A, we can calculate the total amount of data that can be embedded in the image. Of course, the label map needs to be converted into a binary sequence as auxiliary information and embedded in the encrypted image, the purpose of which is to ensure that the original image can be completely reconstructed.

For all the pixels in a natural image, there are 9 kinds of labels, so we need to use 9 binary codes to represent all types of label. Because the number of each label is different, we record the label map through Huffman coding, which can effectively compress the amount of auxiliary information, thereby increasing the payload of the image. Therefore, we predefine 9 kinds of Huffman codes to represent the 9 kinds of labels, namely $\{00, 01, 100, 101, 1100, 1101, 1110, 11110, 11111\}$. We first sort 9 kinds of labels by the number of pixels and we use the shorter code to represent the label with the larger number of pixels. That is, for 9 Huffman codes, where '00' represents the label with the largest number of pixels, and '11111' represents the label with the smallest number of pixels.

For example, in the *Lena* image, the distribution of the label map and the corresponding Huffman coding are shown in Table I, where '-1' represents the reference pixels. We can see that the number of pixels with the tag value equal to 5 in *Lena* is the largest, so these pixels are labeled by '00' code. And the number of pixels with the tag value equal to 1 is the least, so these pixels are labeled by '11111' code.

After the Huffman coding is determined, we can calculate the length of the label map, expressed as LM, and the formula is as follows:

$$LM = \sum_{t=0}^{8} (n_t \times c_t), t = 0, 1, 2, \ldots, 8 \tag{10}$$

where n_t is the number of tag value t in the label map, and c_t is the length of the corresponding Huffman code.

D. Label Map Embedding

In order to generate space for embedding additional data in the encrypted image, we need to embed the label map at first before data hiding operation. In the encrypted image I_e, we first convert the label map into a binary sequence through Huffman coding. Then, we take the Huffman coding rule, the length of the binary sequence, and the binary sequence as auxiliary information. Note that the Huffman encoding rule is 9 Huffman codes, which arranged in the order of 0 to 8 in accordance with the label they represent. Next, in order to ensure that the Huffman coding rule and the label map can be completely extracted in later stages, we need to store the partial auxiliary information in the reference pixels of the first row and column. This is because we need to know the label of the current pixel in advance when extracting auxiliary information, and the current pixel's label can only be obtained from the auxiliary information extracted from the previous pixel, that is, we can't get its tag value directly from the current pixel. Finally, the remaining auxiliary information and the replaced reference pixels are embedded into the encrypted image by multi-MSB substitution according to the label map, and the embedding formula is as follows:

$$x_e{}'(i,j) = \begin{cases} x_e(i,j) mod\, 2^{7-t} + \sum_{s=0}^{t} (b_s \times 2^{7-s}), & 0 \le t \le 6 \\ \sum_{s=1}^{8} (b_s \times 2^{8-s}), & 7 \le t \le 8 \end{cases} \tag{11}$$

Authorized licensed use limited to: National PingTung University -. Downloaded on May 26,2021 at 13:39:47 UTC from IEEE Xplore. Restrictions apply.

TABLE I
Lena'S LABEL MAP DISTRIBUTION AND HUFFMAN CODING

Label	-1	0	1	2	3	4	5	6	7	8
Distribution	1023	9818	9742	15247	33246	44509	53359	41758	24353	29089
Code	-	11110	11111	1110	101	01	00	100	1101	1100

where t is the tag value of the current pixel $x_e(i,j)$ in the I_e, and b_s is auxiliary information to be embedded. After embedding the auxiliary information and the reference pixels, we get the final encrypted image $I_e{}'$ containing the label map. Note that in order to completely extract the auxiliary information in subsequent operations, it is necessary to set the plurality of rows and columns as reference pixels in some rough images.

E. Data Hiding

In this subsection, we need to extract the Huffman coding rule and the label map from the obtained encrypted image $I_e{}'$ before data hiding. First, we extract the partial auxiliary information in the first row and column to obtain the Huffman coding rule and the auxiliary information length. Then, according to the existing auxiliary information and Huffman coding rule, the tag value t of the current pixel is obtained and the $(t+1)$ bits auxiliary information can be extracted. After we have obtained all the auxiliary information, we can restore the label map based on Huffman coding rule. Finally, according to Eq. (11), the additional data is embedded in the remaining pixels, which is the reserved space used to embed the data in the encrypted image. Thus, the marked encrypted image I_{ew} containing additional data is generated. Note that to further enhance the security of the embedded data, we encrypt it by using the data hiding key K_d before embedding process.

F. Data Extraction and Image Recovery

First, a legal receiver can extract the Huffman coding rule and the label map from the marked encrypted image I_{ew}. Then, based on the label map, the reference pixels and the encrypted additional data can be extracted in the same way. Finally, put the reference pixels back to the first row and column. The above work can be done without the key, but the next process will get different results depending on the different key that the receiver owned.

If the recipient only has the data hiding key K_d, the embedded information can be obtained by directly decrypting the extracted encrypted additional data. However, since there is no encryption key, the original image cannot be reconstructed.

If the recipient only has the encryption key K_e, the original image can be recovered without loss. This is because the data hiding key only encrypts the embedded data and does not affect the process of image processing. Firstly, the resulting image is decrypted according to the pseudo-random matrix r generated by K_e, and the process is handled by Eq. (8). Then we get the decrypted image $I_{ew}{}'$, which the front $(t+1)$ bits in each pixel except the reference pixels are different from the original pixel. Because these pixels are embedded with additional information of $(t+1)$ bits according to their tag values t. Next, we scan the pixels in the image except the reference pixels from top to bottom and from left to right. The predicted value $px(i,j)$ of the current pixel $x_{ew}{}'(i,j)$ is calculated using the MED predictor, and then the original pixel $x(i,j)$ can be restored according to the tag value t and the predicted value $px(i,j)$. This is because the t-MSB of $x(i,j)$ is the same as the corresponding $px(i,j)$, and the $(t+1)^{th}$ MSB of $x(i,j)$ can be obtained by negating the $(t+1)^{th}$ MSB of $px(i,j)$. Note that if the tag value is equal to 8, the original pixel is equal to its predicted value. The reconstruction process is expressed as follows:

$$x(i,j) = \begin{cases} px(i,j)^{tMSB} + b_{t+1} \times 2^{7-t} \\ \quad +x_{ew}(i,j) mod\, 2^{7-t} \end{cases}, \ 0 \le t \le 7 \\ px(i,j), \qquad\qquad t = 8 \tag{12}$$

where $px(i,j)^{tMSB}$ is the t-MSB values of $px(i,j)$ obtained by Eq. (7), and b_{t+1} is the $(t+1)^{th}$ binary value of the original pixel obtained according to the following formula:

$$b_{t+1} = \begin{cases} 0\ , \ px^{t+1}(i,j) = 1 \\ 1\ , \ px^{t+1}(i,j) = 0 \end{cases} \tag{13}$$

where $px^{t+1}(i,j)$ is the $(t+1)^{th}$ binary bit of $px(i,j)$ obtained by Eq. (6). Finally, the original image without error is obtained.

Therefore, only there are both the data hiding key K_d and the image encryption key K_e simultaneously, the recipient can reversibly extract the additional data and restore the original image.

IV. EXPERIMENTAL RESULTS AND ANALYSIS

In this section, we first analyze the security of the proposed method, then show the experimental results and make comparison with the state-of-the-art works. Six commonly used test images: *Lena*, *Baboon*, *Jetplane*, *Man*, *Airplane* and *Tiffany*, are shown in Fig. 5. We also test performance of different methods in three databases: BOSSBase [38], BOWS-2 [39], and UCID [40]. The embedding rate (ER) presented by bpp (bits per pixel) is the key indicator. In addition, commonly used metrics: PSNR (Peak signal-to-noise ratio) and SSIM (structural similarity), are adopted to test the reversibility of the method.

A. Security Analysis

We encrypt the original image with a stream cipher, which can hide the feature information of the original image. In order to prove the security of the proposed method, we will analyze it from two aspects: statistical features and probability.

Take the *Lena* image as an example, Fig. 6 is the experimental results of each phase obtained by our method, and Fig. 7 is the histograms corresponding to the image of three stages. For the original image, the histogram containing significant feature

Authorized licensed use limited to: National PingTung University -. Downloaded on May 26,2021 at 13:39:47 UTC from IEEE Xplore. Restrictions apply.

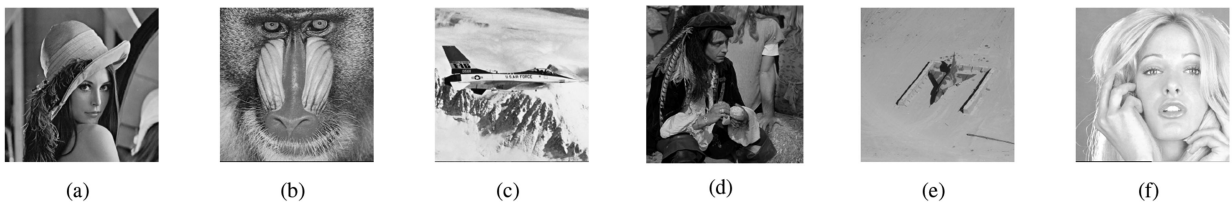


Fig. 5. Test images: (a) $Lena$; (b) $Baboon$; (c) $Jetplane$; (d) Man; (e) $Airplane$; and (e) $Tiffany$.

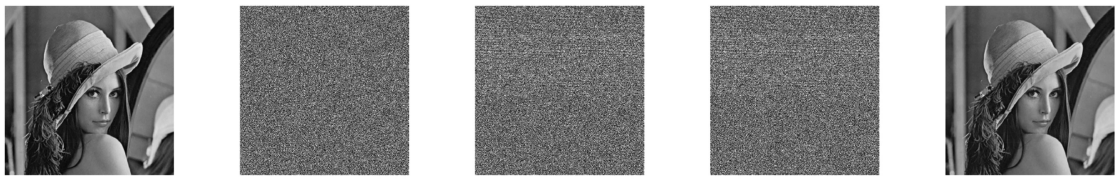


Fig. 6. Experiment with the proposed method, showing the results of each phase: (a) Original image I; (b) Encrypted image I_e; (c) Final encrypted image $I_e{}'$ with label map; (d) Marked encrypted image I_{ew}, with net payload ER $= 2.583$ bpp; (e) Reconstructed image I, PSNR $\rightarrow +\infty$, SSIM $= 1$.

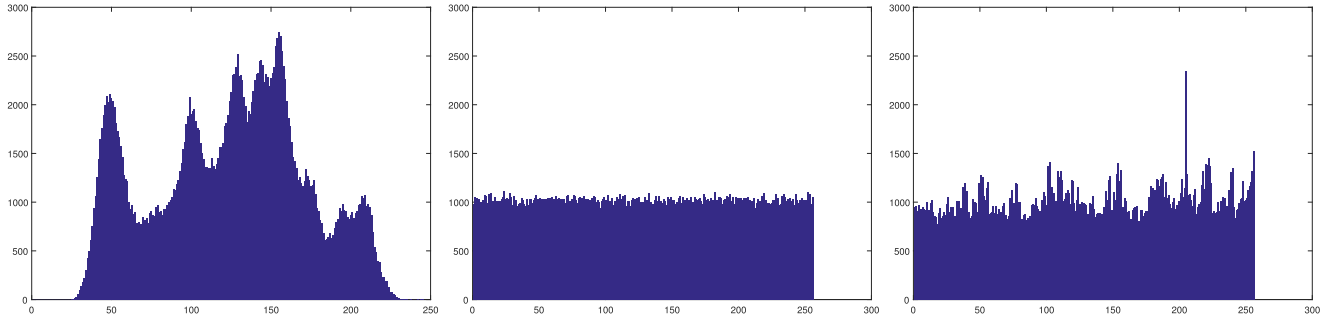


Fig. 7. Histograms of $Lena$ image of three stages: (a) Histogram of the original image I; (b) Histogram of the encrypted image I_e; (c) Histogram of the marked encrypted image I_{ew}.

information is shown in Fig. 7(a). Fig. 7(b) is the histogram of the encrypted image after stream cipher encryption, and the distribution of the pixels becomes uniform so that we cannot obtain useful statistical feature information. After subsequent label map embedding and data embedding, the number of individual pixels in the encrypted image increases sharply but the distribution of overall pixels does not change much. As can be seen from Fig. 7(c), the statistical feature of the image is still significantly different from the original image, and the original content of the image cannot be obtained. This means that our RDHEI method is guaranteed in terms of security.

To further illustrate the security of our approach, we can also calculate the probability of obtaining the original image without the encryption key. For an $M \times N$ grayscale image, we encrypt it with a pseudo-random binary sequence of length $M \times N \times 8$, and each bit in the sequence is likely to be 0 or 1. In other words, this pseudo-random sequence has a total of $2^{M \times N \times 8}$ possibilities, which is a large number. So in the absence of the key, it is almost impossible to obtain a completely correct encryption sequence from so many possibilities. This proves that our method is highly secure, which protects the privacy of the content owner.

B. Performance Analysis

In our method, in order to get accurate prediction values to make the label map more concentrated, we use the MED predictor to find the label map of the image. A concentrated label map means less auxiliary information is recorded, that is, the payload of the image is larger. On the other hand, the use of the MED predictor can also effectively reduce the number of reference pixels and increase the number of embeddable pixels, thereby increasing ER.

After getting the label map, the total EC of an image can be calculated. Similarly, according to the Huffman coding rule and the label map, the length of the auxiliary information can also be calculated, thereby obtaining the size of the net payload. Still take the $Lena$ image as an example, the label distribution and encoding of the pixels are shown in Table II. The first column is 9 types of the label in the $Lena$ image, where '-1' represents the reference pixels. The second column is the number of each label, and the third column is the corresponding Huffman codes. As shown in the fourth column, we can get the amount of data that each pixel can embed, so the total EC of the $Lena$ image can

Authorized licensed use limited to: National PingTung University -. Downloaded on May 26,2021 at 13:39:47 UTC from IEEE Xplore. Restrictions apply.

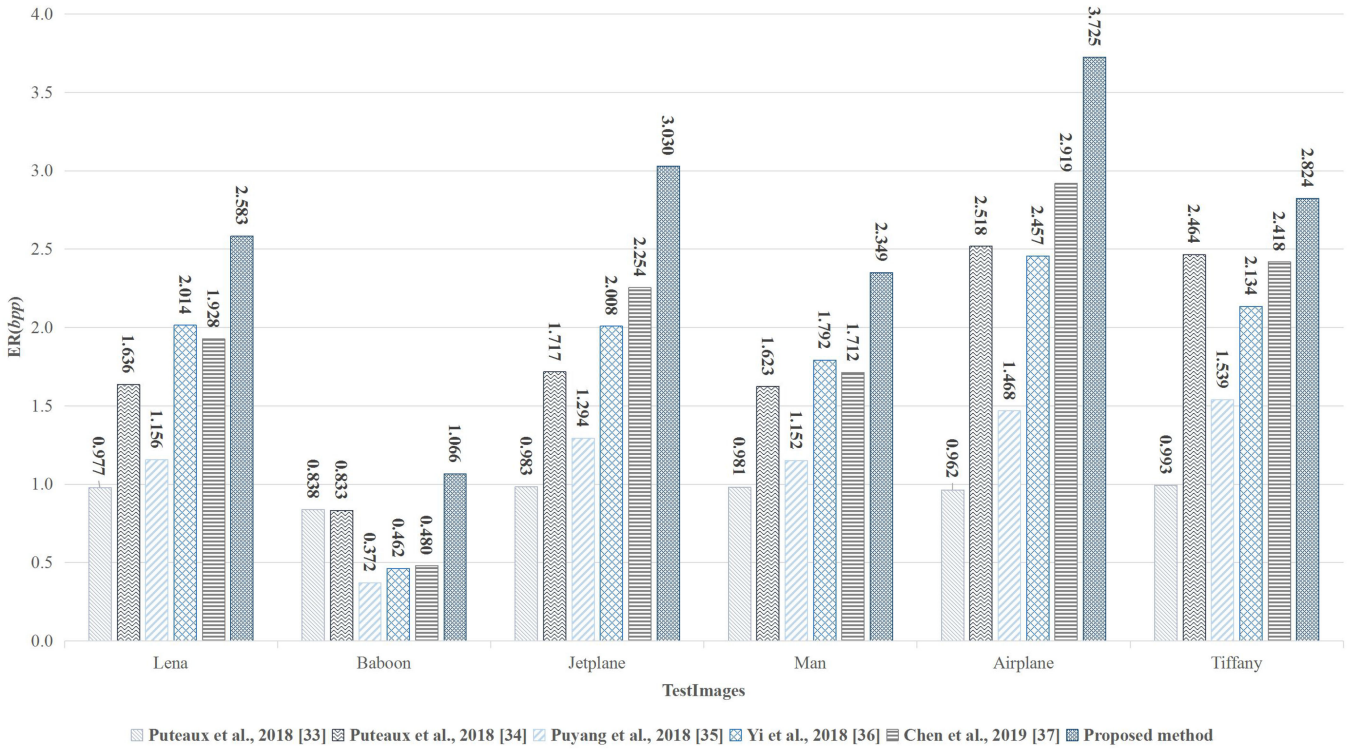


Fig. 8. Comparison of ER (bpp) on six test images.

TABLE II
LABEL HUFFMAN CODING FOR THE *Lena* IMAGE

Label	Distribution	Code	Capacity (bits)	Code length (bits)	Payload (bits)
-1	1023	-	-	-	-
0	9818	11110	1	5	-4
1	9742	11111	2	5	-3
2	15247	1110	3	4	-1
3	33246	101	4	3	1
4	44509	01	5	2	3
5	53359	00	6	2	4
6	41758	100	7	3	4
7	24353	1101	8	4	4
8	29089	1100	8	4	4
Total	-	-	1470568	793304	677264

be got, which is 1470568 bits. Similarly, the fifth column is the Huffman code length of each label, and the total amount of label map converted to a binary sequence is 793304 bits, which is obtained by Eq. (10). Obviously, the total EC minus the amount of binary label map equal to 677264 bits, which can also be calculated from the payload of each pixel in the last column. In addition, we need to subtract the extra 52 bits, of which 32 bits are used to store the Huffman coding rule and 20 bits are used to store the length of the auxiliary information, and then we get the final payload of 677212 bits, which is 2.583 bpp.

Not just the *Lena* image, we also calculated the amount of net payload and auxiliary information for other test images, and the results are shown in Table III.

After theoretically calculating the payload of the image, we can experiment to analyze the feasibility and reversibility of the proposed algorithm. For a 512×512 grey-level images of *Lena*, the original image I is shown in Fig. 6(a), while Fig. 6(b) shows the encrypted image I_e obtained by the encryption key K_e. Then, the content owner embeds the label map into the encrypted image to ensure that the data hider can embed the data in the reserved room, and the final encrypted image I_e' is shown in Fig. 6(c). Fig. 6(d) shows the marked encrypted image I_{ew} after data hiding, whose ER reaches 2.583 bpp. As shown in Fig. 6(e), the image reconstructed according to the encryption key K_e is the same as the original image, that is, the PSNR of the reconstructed image is close to $+\infty$, and the SSIM is equal to 1. In other words, the original image can be reconstructed without loss as long as there is the encryption key. In addition, since the encrypted secret information is extracted together with the Huffman coding rule and the label map, the recipient can directly restore the original data on the basis of the data key K_d.

Furthermore, we analyze our method in three databases [38]–[40]. Among them, BOSSBase [38] and BOWS-2 [39] both have 10,000 grayscale images with size 512×512, while UCID [40] has 1388 grayscale images with size 512×384 or 384×512. For relatively smooth images, all pixels have the larger tag value which more information can be embedded into, and the encoded label map is less, so the payload is large. On the contrary, for rough images, the total EC is smaller and the auxiliary information is larger, so the net payload is smaller.

As shown in Table IV, for images from database BOSSBase, net ER reaches 5.898 bpp in the best case, and only 0.664 bpp in the worst case. Similarly, the best case and the worst case of ER

Authorized licensed use limited to: National PingTung University -. Downloaded on May 26,2021 at 13:39:47 UTC from IEEE Xplore. Restrictions apply.

TABLE III
THE EMBEDDING CAPACITY AND AUXILIARY INFORMATION OF DIFFERENT TEST IMAGES

Test Images	Total EC (bits)	LM (bits)	Extra bits (bits)	Net Payload (bpp)
Lena	1470568	793304	52	2.583
Baboon	1074384	794941	52	1.066
Jetplane	1587880	793441	52	3.030
Man	5584907	3121508	52	2.349
Airplane	1659203	682803	52	3.724
Tiffany	1526934	786527	52	2.824

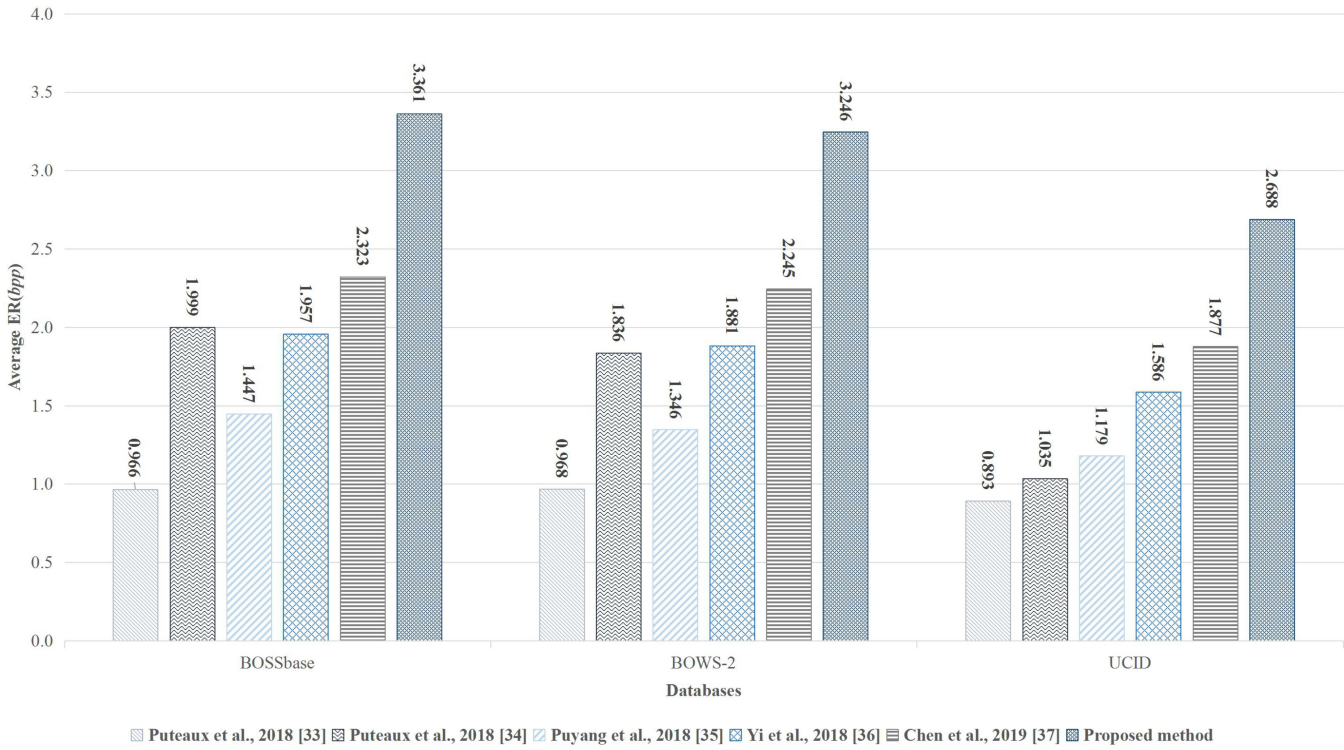


Fig. 9. Average ER (bpp) comparison in different datasets BOSSBase [38], BOWS-2 [39], and UCID [40].

TABLE IV
EXPERIMENTAL RESULTS ON THREE IMAGE DATABASES

Databases	Indicators	Best case	Worst case	Average
BOSSbase	ER (bpp)	5.898	0.664	3.361
	PSNR	$+\infty$	$+\infty$	$+\infty$
	SSIM	1	1	1
BOWS-2	ER (bpp)	5.622	0.628	3.246
	PSNR	$+\infty$	$+\infty$	$+\infty$
	SSIM	1	1	1
UCID	ER (bpp)	5.010	0.397	2.688
	PSNR	$+\infty$	$+\infty$	$+\infty$
	SSIM	1	1	1

from image database BOWS-2 are 5.622 bpp and 0.628 bpp, respectively, while that in the database UCID are 5.010 bpp and 0.397 bpp. In the three databases, the proposed method has an average ER of 3.361 bpp, 3.246 bpp, and 2.688 bpp, respectively. Of course, each image can be recovered error-free, i.e., PSNR $= +\infty$ and SSIM $= 1$. Through the above analysis, it is verified that the proposed method has good performance and can be applied in privacy protection.

C. Comparison With State-of-the-Art Methods

To better demonstrate the performance of our method, we compare the proposed method with five state-of-the-art works [33]–[37]. As shown in Fig. 8, we first compare the ER on the six test images as shown in Fig. 5. The ER of [33] is less than 1 bpp because Puteaux *et al.* embedded only one bit to each embeddable pixel by MSB substitution. In the improved method [34], Puteaux *et al.* performed pixel labeling and data embedding on multiple bit planes of the image, so the ER exceeding 1 bpp. Similarly, Puyang *et al.* [35] also got a higher ER of more than 1 bpp by embedding two bits into each embeddable pixel. However, in these two improved methods, the ER of the $Baboon$ image is not very high. This is because the $Baboon$ image is rough, so that the auxiliary information is excessive, resulting in a decrease in the payload. Therefore, the ER of the $Baboon$

Authorized licensed use limited to: National PingTung University -. Downloaded on May 26,2021 at 13:39:47 UTC from IEEE Xplore. Restrictions apply.

TABLE V
DETAILED INCREMENTS IN TERMS OF NET PAYLOAD (bpp) ON DIFFERENT DATASETS

Databases	Net payload of different methods (*bpp*)						**Increment**
	[33]	[34]	[35]	[36]	[37]	Proposed method	(***bpp***)
BOSSbase	0.966	1.999	1.447	1.957	2.323	3.361	**1.038**
BOWS-2	0.968	1.836	1.346	1.881	2.245	3.246	**1.001**
UCID	0.893	1.035	1.179	1.586	1.877	2.688	**0.811**

image in [36] and [37] is also very low, while the results of other test images are very good. It can be clearly seen from Fig. 8 that the ER of the six test images obtained by our method are higher than these five algorithms, even in the rough image $Baboon$. This is because we can explore the maximum embedding space through multi-MSB prediction and Huffman coding.

As shown in Fig. 9, the experimental results in the three databases [38]–[40] also confirmed that the performance of our method is the best. The average ER of the proposed method in BOSSBase reaches 3.361 bpp, a great improvement in terms of net payload. Similar results are also gained in the other databases BOWS-2 and UCID, with an average ER of 3.246 bpp and 2.688 bpp, respectively. The detailed increments in terms of net payload on different databases are shown in Table V.

V. CONCLUSION

In this paper, a new high-capacity and error-free RDHEI method is proposed based on multi-MSB prediction and Huffman coding. The proposed method can explore larger data embedding space to get higher net payload. This key contribution is achieved based on two techniques: compressing auxiliary information better by using Huffman coding and embedding multi-bits adaptively by multi-MSB substitution. Specifically, in our method, after the tag value of each pixel is obtained based on its predicted value, the total data embedding capacity of an image is determined. The advantage of using Huffman coding is to compress the label map better and free up more space for embedding information.

In future work, we will work to improve algorithm performance in two ways. On the one hand, we are committed to researching more accurate prediction algorithms to make the pixels' tag values more concentrated, thus increasing the embedding capacity. On the other hand, for different images, we can choose the optimal 9 Huffman codes as the mapping rules, so that the length of the binary label map is reduced, that is, the performance of the algorithm is improved.

REFERENCES

[1] H. Zhou, K. Chen, W. Zhang, Y. Yao, and N. Yu, "Distortion design for secure adaptive 3D mesh steganography," *IEEE Trans. Multimedia*, vol. 21, no. 6, pp. 1384–1398, Nov. 2018.

[2] G. Coatrieux, C. Le Guillou, J.-M. Cauvin, and C. Roux, "Reversible watermarking for knowledge digest embedding and reliability control in medical images," *IEEE Trans. Inf. Technol. Biomed.*, vol. 13, no. 2, pp. 158–165, Mar. 2009.

[3] S. Lee, C. D. Yoo, and T. Kalker, "Reversible image watermarking based on integer-to-integer wavelet transform," *IEEE Trans. Inf. Forensics Secur.*, vol. 2, no. 3, pp. 321–330, Sep. 2007.

[4] R. Y. M. Li, O. C. Au, C. K. M. Yuk, S.-K. Yip, and T.-W. Chan, "Enhanced image trans-coding using reversible data hiding," in *Proc. IEEE Int. Symp. Circuits. Syst.*, May 2007, pp. 1273–1276.

[5] W. Jiang, H. Zhou, W. Zhang, and N. Yu, "Reversible data hiding in encrypted 3D mesh models," *IEEE Trans. Multimedia*, vol. 20, no. 1, pp. 55–67, Jul. 2017.

[6] M. J. Weinberger, G. Seroussi, and G. Sapiro, "The LOCO-I lossless image compression algorithm: Principles and standardization into JPEG-LS," *IEEE Trans. Image Process.*, vol. 9, no. 8, pp. 1309–1324, Aug. 2000.

[7] M. U. Celik, G. Sharma, A. M. Tekalp, and E. Saber, "Lossless generalized-LSB data embedding," *IEEE Trans. Image Process.*, vol. 14, no. 2, pp. 253–266, Feb. 2005.

[8] M. U. Celik, G. Sharma, and A. M. Tekalp, "Lossless watermarking for image authentication: A new framework and an implementation," *IEEE Trans. Image Process.*, vol. 15, no. 4, pp. 1042–1049, Apr. 2006.

[9] Z. Ni, Y.-Q. Shi, N. Ansari, and W. Su, "Reversible data hiding," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 16, no. 3, pp. 354–362, Mar. 2006.

[10] X. Li, B. Li, B. Yang, and T. Zeng, "General framework to histogram-shifting-based reversible data hiding," *IEEE Trans. Image Process.*, vol. 22, no. 6, pp. 2181–2191, Jun. 2013.

[11] X. Li, W. Zhang, X. Gui, and B. Yang, "Efficient reversible data hiding based on multiple histograms modification," *IEEE Trans. Inf. Forensics Secur.*, vol. 10, no. 9, pp. 2016–2027, Sep. 2015.

[12] J. Wang, J. Ni, X. Zhang, and Y. Shi, "Rate and distortion optimization for reversible data hiding using multiple histogram shifting," *IEEE Trans. Cybern.*, vol. 47, no. 2, pp. 315–326, Feb. 2017.

[13] J. Tian, "Reversible data embedding using a difference expansion," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 13, no. 8, pp. 890–896, Aug. 2003.

[14] B. Ou, X. Li, Y. Zhao, R. Ni, and Y. Shi, "Pairwise prediction-error expansion for efficient reversible data hiding," *IEEE Trans. Image Process.*, vol. 22, no. 12, pp. 5010–5021, Dec. 2013.

[15] I.-C. Dragoi and D. Coltuc, "Local-prediction-based difference expansion reversible watermarking," *IEEE Trans. Image Process.*, vol. 23, no. 4, pp. 1779–1790, Apr. 2014.

[16] X. Zhang, Y. Ren, L. Shen, Z. Qian, and G. Feng, "Compressing encrypted images with auxiliary information," *IEEE Trans. Multimedia*, vol. 16, no. 5, pp. 1327–1336, Apr. 2014.

[17] W. Puech, M. Chaumont, and O. Strauss, "A reversible data hiding method for encrypted images," *Proc. SPIE*, vol. 6819, p. 6819E, Mar. 2008.

[18] X. Zhang, "Reversible data hiding in encrypted image," *IEEE Signal Process. Lett.*, vol. 18, no. 4, pp. 255–258, Apr. 2011.

[19] W. Zhang, H. Wang, D. Hou, and N. Yu, "Reversible data hiding in encrypted images by reversible image transformation," *IEEE Trans. Multimedia*, vol. 18, no. 8, pp. 1469–1479, May 2016.

[20] Y. Zhou, L. Bao, and C. P. Chen, "A new 1D chaotic system for image encryption," *Signal Process.*, vol. 97, pp. 172–182, 2014.

[21] Z. Hua, Y. Zhou, C.-M. Pun, and C. L. P. Chen, "2D sine logistic modulation map for image encryption," *Inf. Sci.*, vol. 297, pp. 80–94, 2015.

[22] Z. Hua, F. Jin, B. Xu, and H. Huang, "2D Logistic-sine-coupling map for image encryption," *Signal Process.*, vol. 149, pp. 148–161, 2018.

[23] W. Hong, T. S. Chen, and H. Y. Wu, "An improved reversible data hiding in encrypted images using side match," *IEEE Signal Process. Lett.*, vol. 19, no. 4, pp. 199–202, Apr. 2012.

[24] X. Zhang, "Separable reversible data hiding in encrypted image," *IEEE Trans. Inf. Forensics Secur.*, vol. 7, no. 2, pp. 826–832, Apr. 2012.

[25] X. Wu and W. Sun, "High-capacity reversible data hiding in encrypted images by prediction error," *Signal Process.*, vol. 104, pp. 387–400, Nov. 2014.

[26] Z. Qian and X. Zhang, "Reversible data hiding in encrypted images with distributed source encoding," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 26, no. 4, pp. 636–646, Apr. 2016.

Authorized licensed use limited to: National PingTung University -. Downloaded on May 26,2021 at 13:39:47 UTC from IEEE Xplore. Restrictions apply.

[27] Z. Qian, X. Zhang, and S. Wang, "Reversible data hiding in encrypted JPEG bitstream," *IEEE Trans. Multimedia*, vol. 16, no. 5, pp. 1486–1491, Apr. 2014.
[28] K. Ma *et al.*, "Reversible data hiding in encrypted images by reserving room before encryption," *IEEE Trans. Inf. Forensics Secur.*, vol. 8, no. 3, pp. 553–562, Mar. 2013.
[29] W. Zhang, K. Ma, and N. Yu, "Reversibility improved data hiding in encrypted images," *Signal Process.*, vol. 94, pp. 118–127, Jan. 2014.
[30] X. Zhang, J. Long, Z. Wang, and H. Cheng, "Lossless and reversible data hiding in encrypted images with public-key cryptography," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 26, no. 9, pp. 1622–1631, Sep. 2016.
[31] D. Xu and R. Wang, "Separable and error-free reversible data hiding in encrypted images," *Signal Process.*, vol. 123, pp. 9–21, Jun. 2016.
[32] F. Huang, J. Huang, and Y. Q. Shi, "New framework for reversible data hiding in encrypted domain," *IEEE Trans. Inf. Forensics Secur.*, vol. 11, no. 12, pp. 2777–2789, Dec. 2016.
[33] P. Puteaux and W. Puech, "An efficient MSB prediction-based method for high-capacity reversible data hiding in encrypted images," *IEEE Trans. Inf. Forensics Secur.*, vol. 13, no. 7, pp. 1067–1681, Jan. 2018.
[34] P. Puteaux and W. Puech, "EPE-based huge-capacity reversible data hiding in encrypted images," in *Proc. IEEE Int. Workshop Inf. Forensics Secur.*, Dec. 2018, pp. 1–7.
[35] Y. Puyang, Z. Yin, and Z. Qian, "Reversible data hiding in encrypted images with two-MSB prediction," in *Proc. IEEE Int. Workshop Inf. Forensics Secur.*, Dec. 2018, pp. 1–7.
[36] S .Yi and Y. Zhou, "Separable and reversible data hiding in encrypted images using parametric binary tree labeling," *IEEE Trans. Multimedia*, vol. 21, no. 1, pp. 51–64, Jan. 2019.
[37] K. Chen and C. C. Chang, "High-capacity reversible data hiding in encrypted images based on extended run-length coding and block-based MSB plane rearrangement," *J. Vis. Commun. Image Representation*, vol. 58, pp. 334–344, 2019.
[38] P. Bas, T. Filler, and T. Pevny, "Break our steganographic system The ins and outs of organizing BOSS," in *Proc. 13th Int. Workshop Inf. Hiding*, May 2011, pp. 59–70. Available: http://dde.binghamton.edu/download/
[39] P. Bas and T. Furon, "Image database of BOWS-2," 2017. [Online]. Available: http://bows2.ec-lille.fr. Accessed: Jun. 20, 2017.
[40] G. Schaefer and M. Stich, "UCID: An uncompressed color image database," in *Proc. SPIE*, vol. 5307, pp. 472–480, 2003. [Online]. Available: http://vision.doc.ntu.ac.uk/



Zhaoxia Yin received the B.Sc., M.E. and Ph.D. degrees from Anhui University, Hefei, China, in 2005, 2010, and 2014, respectively. She is currently working as an Associate Professor and a Ph.D advisor with the School of Computer Science and Technology, Anhui University. She is also the Principal Investigator of two NSFC Projects. Her primary research focus including information hiding, multimedia security and she has authored many SCI/EI indexed papers in journals, edited books, and refereed conferences. She is the IEEE/ACM/CCF member, CSIG senior member and the Associate Chair of the academic committee of CCF YOCSEF Hefei 2016–2017.



Youzhi Xiang received the bachelor's degree in computer science and technology in 2017 and currently he is working toward the master's degree with the School of Computer Science and Technology, Anhui University, Hefei, China. His current research interests include reversible data hiding in encrypted images.



Xinpeng Zhang received the B.S. degree in computational mathematics from Jilin University, Changchun, China, in 1995, and the M.E. and Ph.D. degrees in communication and information system from Shanghai University, Shanghai, China, in 2001 and 2004, respectively. Since 2004, he had been with the faculty of the School of Communication and Information Engineering, Shanghai University, and he is currently a Professor of School of Computer Science, Fudan University, Shanghai, China. He was with the State University of New York, Binghamton as a visiting scholar from January 2010 to January 2011, and Konstanz University as an experienced researcher sponsored by the Alexander von Humboldt Foundation from March 2011 to May 2012. His research interests include multimedia security, image processing, and digital forensics. He has authored more than 200 papers in these areas. He served the IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY as an Associate Editor from 2014 to 2017.

Authorized licensed use limited to: National PingTung University -. Downloaded on May 26,2021 at 13:39:47 UTC from IEEE Xplore. Restrictions apply.